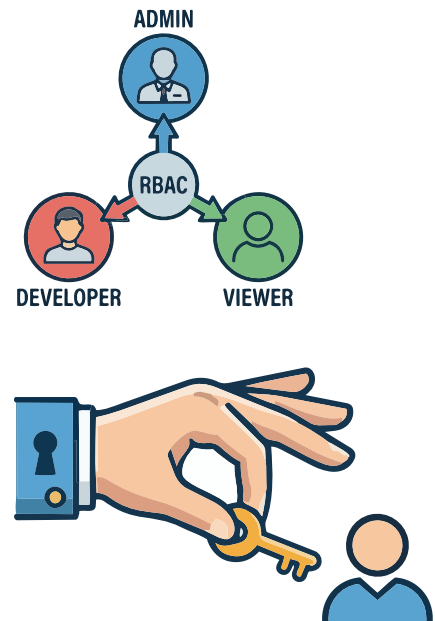
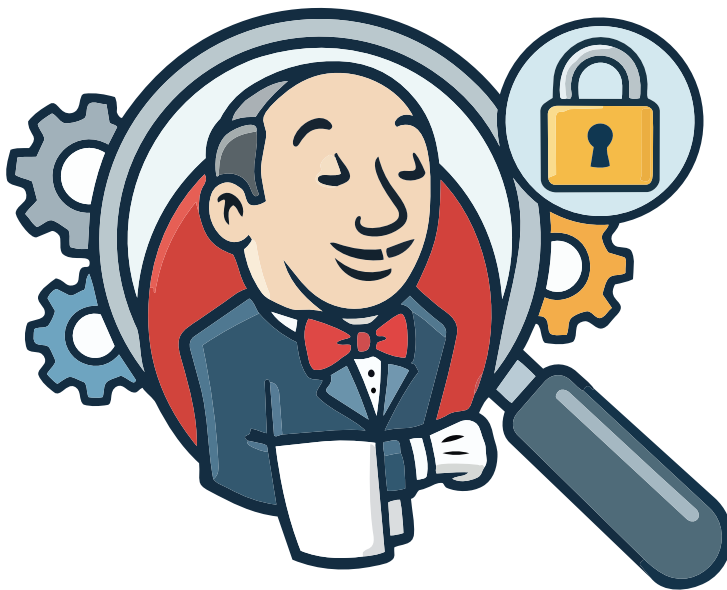


Hardening Jenkins: A Deep Dive into Role-Based Access Control

Role-Based Access Control, a model that allows access to systems based on the job role, is essential for the security of an organisation.



In many fast-moving teams, it is tempting to give every member 'Admin' access to the Jenkins server. It seems convenient at first — no one gets blocked and everyone can fix their own builds. However, in a professional environment, this is a major security risk.

When everyone is an Admin, one accidental click can delete a production pipeline or expose sensitive company passwords. Furthermore, if a single developer's account is compromised, the attacker instantly gains 'the keys to the kingdom', allowing them to tamper with code or move into other company systems.

Role-Based Access Control, or

RBAC, is a security model that gives people access based on their job role rather than their individual name.

Instead of deciding what 'Arun' can do, we decide what a 'Developer' can do. We create a role with specific permissions and then simply add users to that role. This follows the 'Principle of Least Privilege', which means giving users exactly what they need to do their job, and nothing more.

Pre-requisites: Setting the stage

Before securing your environment, ensure you have a clean, working installation of Jenkins. For this guide, we are focusing on a local installation that mimics a corporate setup.

- **Java runtime:** Jenkins is a Java-based application and requires a supported version like Java 17 or 21.
- **Jenkins installation:** For a local environment, you can use the Windows MSI installer to run Jenkins as a Windows service.
 - *Pro tip:* If you have other Java applications running on the default port (8080), change your Jenkins port to 8081 during the installation wizard to avoid conflicts.
- **Maven and JDK path:** Ensure Maven is installed and its path is globally recognised in your system variables so that Jenkins can trigger your builds later.